Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

RUTGERS
UNIVERSITY

Course Name: Computer Architecture

Course Number and Section: **14:332:333:02**

Experiment: [Experiment # [6] – 6 CPU Structure & Pipelining Datapath]

Lab Instructor: Xin Li

Date Performed: 11/22/2024

Date Submitted: 12/13/2024

Submitted by: Samuel Habib 217007500



Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

Computer Architecture and Assembly Lab (14:332:333)
Summer 2024

Lab 6 *CPU Structure & Pipelining Datapath*

Goals

After completing lab 6, you will:

- Know the difference between pipelined and non-pipelined processors
- Learn how the pipeline works; know how to prevent hazards in the pipeline

Introduction

A CPU has 5 basic components, the Arithmetic and Logic Unit (ALU), the Instruction Memory, the Register File, the Data Memory, and its Control block. The ALU is responsible for executing the arithmetic functions, such as addition, subtraction, and the logical functions, e.g., AND, OR, and logical shift, as well as computing branch, jump, and memory addresses. The instruction memory is the memory where an assembly program is stored before execution. The Program Counter contains the address of the instruction being fetched. The data memory is the memory where all the data produced or not are stored. Finally, the control block is responsible for decoding instructions and setting all necessary control signals with the proper values. Figure 1 shows a diagram of a simple CPU. Computers execute instructions in a serial manner. A way to speed up a program's execution is by using a Pipeline. Pipelining is a method to increase the throughput of the executing instruction without actually affecting the time a single instruction needs to execute. Finally, control is the most challenging aspect of processor design: it is both the hardest part to get right and the hardest part to make fast. One of the difficult parts of control design is implementing *exceptions* and *interrupts*. An exception is an unscheduled event that disrupts program execution. An interrupt is an exception that comes from outside of the processor, such as when an Input/Output device needs servicing.

CPU Structure

In order to examine the structure of the CPU, the implementation of actual instructions must be analyzed, so that the organization of the memory units, the ALU, and the control of the CPU are understood. Figure 1 depicts the parts of a CPU. The most important parts are:

1. The Program Counter (PC) points to the address of the instruction that is being fetched from the instruction memory.

2. The Register File contains registers that store the state arguments for the ALU or results of Arithmetic operations, at every clock cycle.

3. The Arithmetic and Logic Unit (ALU) is responsible for executing every arithmetic or logical operation that is needed by an instruction.

4. The Data Memory contains all the data that either is generated by the execution of the instructions or are the initial data that a program may need. 5. The Instruction Memory holds the compiled and linked instructions before execution.

6. The Control Block of the processor decodes each instruction and sets all the control signals according to that instruction.

7. The ALU control is responsible for passing the correct control signal to the ALU according to the arithmetic or logical operation that is requested by the Control unit.

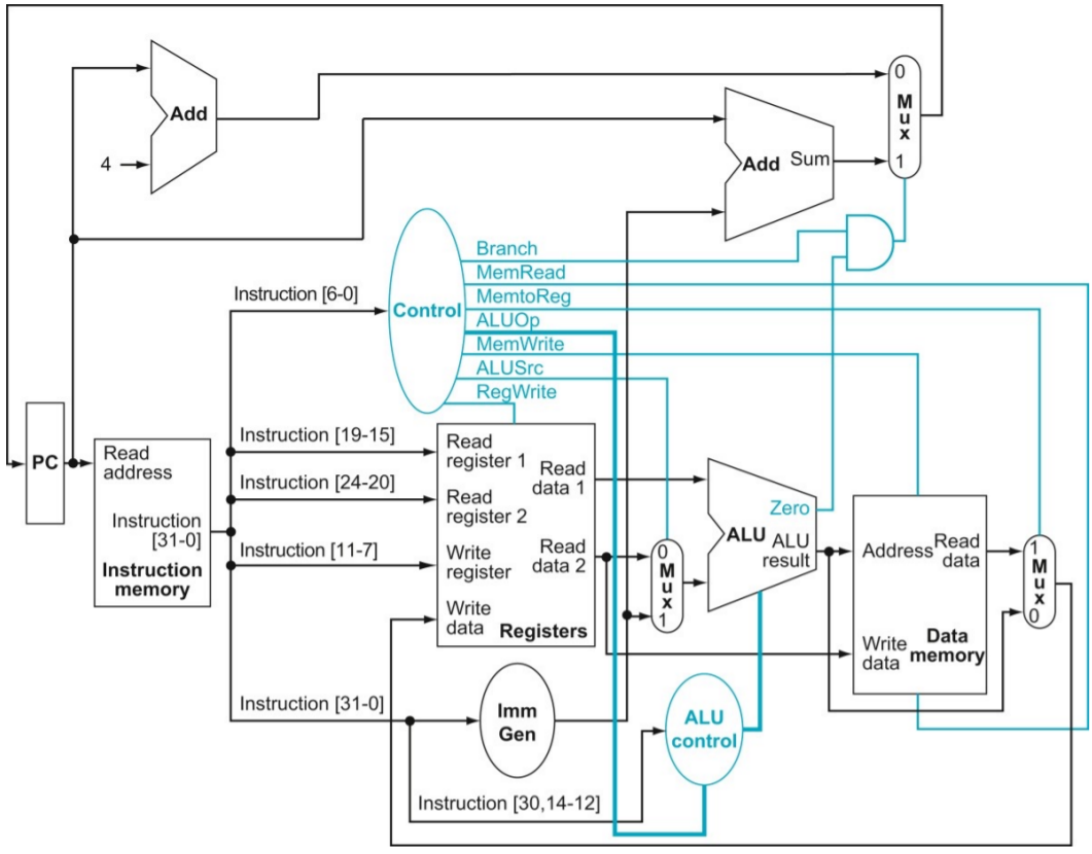


Figure 1. The simple data path with the control unit is shown.

In order to understand how an instruction is executed and how the data path operates, two

examples will be given below. The first example explains the instruction **add x5, x6, x7** and the second explains the instruction **sw x5, offset(x6)**. Table 1 shows the values the control signals have for the two examples.

Example 1

add x5, x6, x7

The execution can be shown in four steps:

1. The instruction is fetched and the PC is incremented by 4.
2. Two registers, **x6** and **x7**, are read from the register file. Also, the main control unit sets of the control lines for an add operation.
3. The ALU operates on the data read from the register file, using the function code (bits: 2 ALUOp of the instruction) to generate the ALU function.
4. The result of the ALU is written back into the register file using the bits 15:11 of the instruction to select the destination register **x5.**

Example 2

sw x5, offset(x6)

The execution can be shown in 5 steps [1]:

1. The instruction is fetched and the PC is incremented.
2. A register (**x5**) value is read from the register file.
3. The ALU computes the sum of the value read from the register file and the sign-extended, lower 16 bits of the instruction (**offset**).
4. The sum of the ALU is used as the address for the data memory.
5. The data from the register file is written into the memory unit; the register destination is given by **x6**.

Instruction	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp [1:0]
`add x5, x6, x7`	0	0	1	0	0	0	10
`sw x5, offset(x6)`	1	x	0	0	1	0	00

Table 1. The setting of the control lines of examples 1 and 2 based on Figure 1.

Signal Name	Explanation
ALUSrc	Controls the MUX to select the second input of the ALU either from the second output of the register file or the sign extended lower 16 bits of the instruction
MemtoReg	Selects the data fed to the register Write data input port either from the ALU output or the data memory output
RegWrite	The register on the Write register input is written with the value on the Write data input
MemRead	Data memory contents designated by the address input are put on the Read data output
MemWrite	Data memory contents designated by the address input are replaced by the value on the Write data input
Branch	Line goes HI if the instruction is a branch instruction
ALUOp[1:0]	The ALU operation.

Pipeline Programming and Hazards

When a processor supports pipeline, an instruction can be divided into four or five steps which are 1) Instruction Fetch [IF], 2) Instruction Decode [ID], 3) Execution [EX], 4) Data Memory Access [MEM] and 5) Write Back the result into the register file [WB]. For a processor to achieve its maximum throughput, the pipeline must always be kept full (i.e. each stage of the pipeline needs to be occupied at each clock cycle). Also, the clock cycle time is kept fixed and is defined by the slowest step of instruction.

Let us now see an example. Consider 3 consecutive instructions A, B, and C. Each instruction has the above-mentioned five steps. When a processor does not support pipelining, each instruction will be executed after the previous one has finished, as it is shown in Table 2. Also, suppose that instruction A starts at cycle 1.

Instr/Cycle	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

A	IF	ID	EX	MEM	WB										
B						IF	ID	EX	MEM	WB					
C											IF	ID	EX	MEM	WB

Table 2. Instruction execution without pipelining

On the other hand, when the pipeline is supported, each instruction can start executing even though the previous one has not necessarily finished.

Instr/Cycle	1	2	3	4	5	6	7	8	9	10	11	12
A	IF	ID	EX	MEM	WB							
B		IF	ID	EX	MEM	WB						
C			IF	ID	EX	MEM	WB					

Table 3. Instruction execution with pipeline support

Note that there are situations in pipelining when the next instruction cannot execute in the following clock cycle. These events are called Hazards and there are three different types of hazards:

A. **Structural Hazard**. When a planned instruction cannot execute in the proper clock cycle because the hardware does not support the combination of instructions that are set to execute.

B. **Data Hazard or Pipeline Data Hazard**. When a planned instruction cannot execute in the proper clock cycle because data that is needed to execute the instruction is not yet available.

C. **Control Hazard or Branch Hazard**. When the instruction cannot execute in the proper pipeline clock cycle because the instruction that was fetched is not the one that is needed; that is, the flow of instruction addresses is not what the pipeline expected.

To understand a Data Hazard, consider the following code fragment:

```
lw x5, 0(x6)
```



Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

```
sub x7, x5, x6
```

The execution of the code with pipeline support is as follows:

Instr/Cycle	1	2	3	4	5	6
lw x5,0(x6)	IF	ID	EX	MEM	WB	
sub x7,x5,x6		IF	ID	EX	MEM	WB

The **lw** will write the value from the memory in the register file in step WB, in the meantime, the **sub** instruction will have retrieved its operands in the ID step. This actually means that the old value of register **x5** will be used for the addition.

Such hazards can be prevented by either writing a program that does not provoke any hazard or by adding an ample number of no-operation instructions (NOP) bubbles between the instructions that depend on the result of each other. NOP operations act like bubbles inside the pipeline, delaying the execution of the second instruction (and subsequent instructions) until the values it requires as input are ready. When the NOP technique is used in the above example, the following happens:

Instr/Cycl	1	2	3	4	5	6	7	8
lw x5, 0(x6)	IF	ID	EX	MEM	WB			
sub x7,x5,x6		IF	Stall	Stall	ID	EX	MEM	WB

Unfortunately, ***Stalling*** slows down the pipeline and degrades its efficiency. An alternative technique instead of NOPs is ***Forwarding***. In this technique, the result of an instruction will be sent forward to the next instruction before it is written back to the register file and it is possible that stalling is not needed. The above data hazard between **lw** and **sub** can be resolved with forwarding:

Instr/Cycle	1	2	3	4	5	6	7	8

lw x5, 0(x6)	IF	ID	EX	MEM	WB			
sub x7, x5, x6		IF	Stall	ID	EX	MEM	WB	

It can be easily seen that a Stall cycle still exists. The reason is that the **lw** instruction will not have the data ready for forwarding before it is read from the memory. When it writes back the value in the register file, it also forwards it to the input of the ALU for immediate use (as symbolized by the arrow). If that stall does not exist even with forwarding, the forwarded result would arrive too late and the result of the **sub** instruction would be wrong.

The third technique that a programmer can use to avoid data hazards is ***Instruction Reordering***. As implied by its name, the independence between instructions can be exploited in order to reorder them in a way that keeps the pipeline as full as possible.

Consider the following code segment:

```
lw x5, 0(x6)
addi x7, x5, 2
sub x28, x29, x9
```

Without instruction reordering, the execution will be the following:

Instr/Cycle	1	2	3	4	5	6	7	8	9
lw x5,0(x6)	IF	ID	EX	MEM	WB				
addi x7,x5,2		IF	Stall	Stall	ID	EX	MEM	WB	
sub x28,x29,x9					IF	ID	EX	MEM	WB

It is clear that two cycles are lost due to the NOP instruction. Also, it is easily seen that **sub** does not need the result of the **lw** or the **add**. In this case, **sub** is independent of the other two and it can be moved between them. After reordering the execution will look like this:



Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

Instr/Cycle	1	2	3	4	5	6	7	8	9
lw x5,0(x6)	IF	ID	EX	MEM	WB				
sub x28,x29,x9		IF	ID	EX	MEM	WB			
addi x7,x5,2			IF	Stall	ID	EX	MEM	WB	

Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

Exercise 1 DataPath Control [30 pts]

Suppose that we have the CPU in Figure 1 and we use the CPU to execute the following program.

```
lw  x5, 0(x10)
lw  x6, 4(x10)
add x7, x6,   x5
sw  x7, 8(x10)
beq x7, x0, Lab6
or x7, x5,   x6
...
Lab6:
```

Please indicate the values of the control signals for each instruction in the given program by filling in the following table.

Instruction	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp[1:0]
lw	1	1	1	1	0	0	00
add	0	0	1	0	0	0	10
sw	1	0	0	0	1	0	00
beq	0	X	0	0	0	1	01
or	0	0	1	0	0	0	11



Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

Exercise 2 Pipeline & Hazards [30 pts]

Please read the program in Exercise 1 to answer the following questions:

2.1 [6 pts] Suppose that register **x10** has the initial values of **0x10000020**. The values stored in **0x10000020** and **0x10000024** are **3** and **9** respectively. Please Fill in the following table with the program's step-by-step result.

	x5	x6	x7
`lw x5, 0(x10)`	3	NULL	NULL
`lw x6, 4(x10)`	3	9	NULL
`add x7, x6, x5`	3	9	12
`sw x7, 8(x10)`	3	9	12
`beq x7, x0, Lab6`	3	9	12
`or x7, x5, x6`	3	9	12

2.2 [6 pts] We can assume that the processor supports the 5-stage pipeline. Please identify all the hazards in the original program and explain what types of hazards they are.

- lw x6, 4(x10) → add x7, x6, x5: x6 is loaded in lw but is needed in add before it's ready.
- add x7, x6, x5 → sw x7, 8(x10): x7 is updated in add but is needed in sw before it's ready.
- sw x7, 8(x10) → beq x7, x0, Lab6: x7 is written in sw at the same time beq needs to read it.

2.3 [6 pts] Please resolve all the hazards (if possible) through stalling. Please draw an execution table to explain your answer.

2.4 [6 pts] Please resolve all the hazards (if possible) by forwarding. Can all the hazards be resolved through forwarding only? Please provide your reasoning. Please draw an execution table to explain your answer.



Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

2.5 [6 pts] Please resolve the hazards (if possible) by instructions reordering. Can all the hazards be addressed through instruction reordering only? Please provide your reasoning. Please draw an execution table to explain your answer.

Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

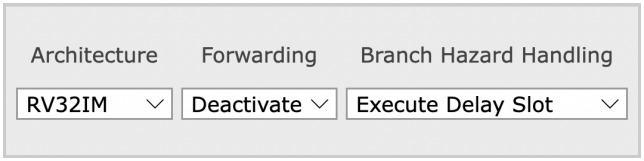
Exercise 3 WebRISC-V Simulator [40 pts]

Simulator Website: https://webriscv.dii.unisi.it/index.php

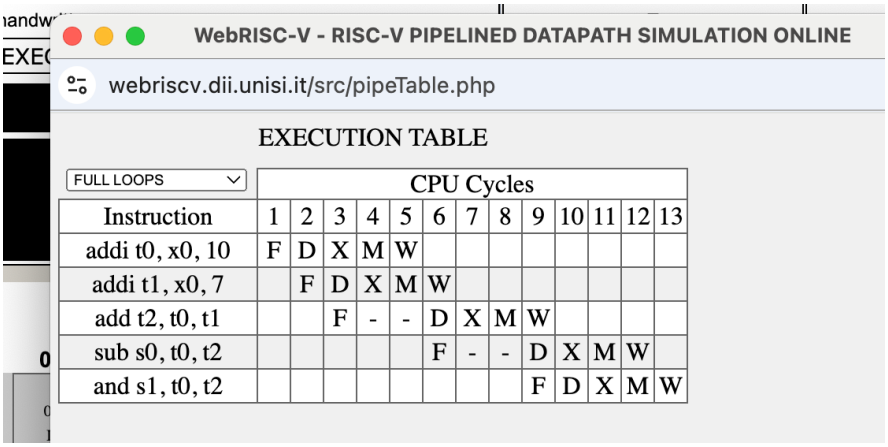
3.1 [20 pts] Please read the following RISC-V assembly code:

```
addi x5, x0, 10
addi x6, x0,  7
add  x7, x5, x6
sub  x8, x5, x7
and  x9, x5, x7
```

3.1.1 [4 pts] Please execute the program using the WebRISC-V simulator by applying the following settings:



How many cycles would it take for the program to execute? Please include a screenshot of the execution table in your report.



Instruction	1	2	3	4	5	6	7	8	9	10	11	12	13
addi t0, x0, 10	F	D	X	M	W								
addi t1, x0, 7		F	D	X	M	W							
add t2, t0, t1			F	-	-	D	X	M	W				
sub s0, t0, t2						F	-	-	D	X	M	W	
and s1, t0, t2									F	D	X	M	W

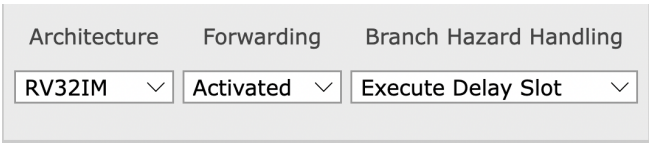
13 cycles

3.1.2 [8 pts] Please identify all the hazards in **3.1.1** and explain.

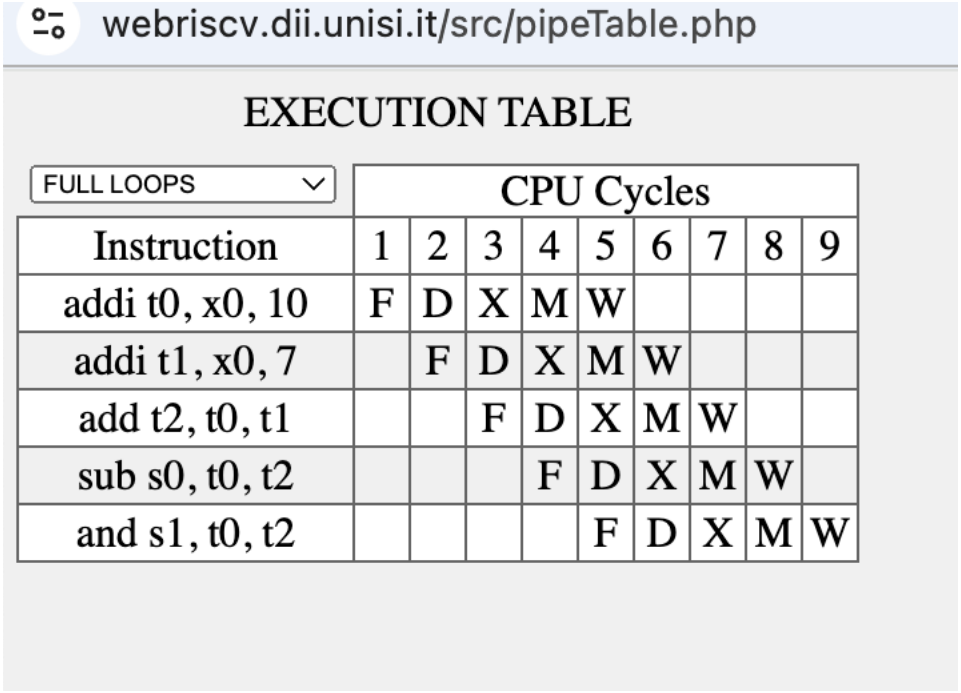
Data hazard between *addi x6, x0, 7* and add x7, x5, x6 because x6 is used as the destination in addi

and then immediately it's used as a source in add. The same type of data hazard between *sub x8, x5, x7 and* and x9, x5, x7.

3.1.3 [8 pts] Please execute the program using the WebRISC-V simulator by applying the following settings:



How many cycles would it take for the program to execute? Please: (1) include a screenshot of the execution table in your report; (2) explain how the hazards are resolved.



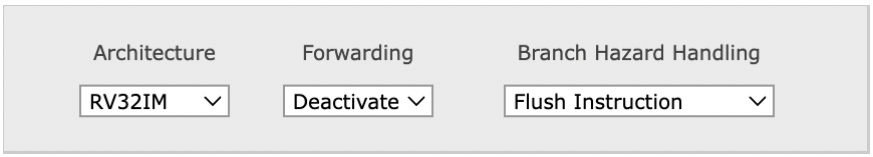
9 cycles, forwarding makes it so the pipeline is able to bypass the register file and route directly from Execution or Memory. In the third line to fourth, instead of waiting for the W, add t2, t0, t1, gets the value directly from the previous execution. The same principle applies for the second hazard.

Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

3.2 [20 pts] Please read the following RISC-V assembly code:

```
addi sp, sp, -12  #store the array {1,2,3} in the stack
addi t0, x0,   1
sw   t0, 8(sp)
addi t0, x0,   2
sw   t0, 4(sp)
addi t0, x0,   3
sw   t0, 0(sp)
addi a1, x0,   3  #the size of the array
addi a2, x0,   0
j    loop
loop:             #compute the sum of the array
bge  a2, a1, end
lw   t0, 0(sp)
addi sp, sp,   4
add  a0, a0,  t0
addi a2, a2,   1
j    loop
end:
```

3.2.1 [4 pts] Please execute the program using the WebRISC-V simulator with the following settings:



Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

EXECUTION TABLE

FULL LOOPS

Instruction	CPU Cycles 1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47
addi sp, sp, -12	F	D	X	M	W																																										
addi t0, x0, 1		F	D	X	M	W																																									
sw t0, 8(sp)			F	-	-	D	X	M	W																																						
addi t0, x0, 2						F	D	X	M	W																																					
sw t0, 4(sp)							F	-	-	D	X	M	W																																	
addi t0, x0, 3										F	D	X	M	W																															
sw t0, 0(sp)											F	-	-	D	X	M	W																											
addi a1, x0, 3														F	D	X	M	W																											
addi a2, x0, 0															F	D	X	M	W																										
addi a0, x0, 0																F	D	X	M	W																									
jal x0, 8																	F	D	X	M	W																								
bge a2, a1, 48																		F																											
bge a2, a1, 48																			F	D	X	M	W																						
lw t0, 0(sp)																				F	D	X	M	W																				
addi sp, sp, 4																					F	D	X	M	W																			
add a0, a0, t0																						F	-	D	X	M	W																	
addi a2, a2, 1																								F	D	X	M	W																
jal x0, -40																									F	D	X	M	W															
bge a2, a1, 48																											F	D	X	M	W													
lw t0, 0(sp)																												F	D	X	M	W												
addi sp, sp, 4																													F	D	X	M	W											
add a0, a0, t0																														F	-	D	X	M	W									
addi a2, a2, 1																																F	D	X	M	W								
jal x0, -40																																	F	D	X	M	W							
bge a2, a1, 48																																			F	D	X	M	W					
lw t0, 0(sp)																																				F	D	X	M	W				
addi sp, sp, 4																																					F	D	X	M	W			
add a0, a0, t0																																						F	-	D	X	M	W		
addi a2, a2, 1																																							F	D	X	M	W		
jal x0, -40																																								F	D	X	M	W	
bge a2, a1, 48																																										F	D	X	M	W
lw t0, 0(sp)																																											F			

How many cycles would it take for the program to execute? Please include a screenshot of the execution table in your report.

3.2.2 [8 pts] Please identify all the hazards in **3.2.1** and explain.

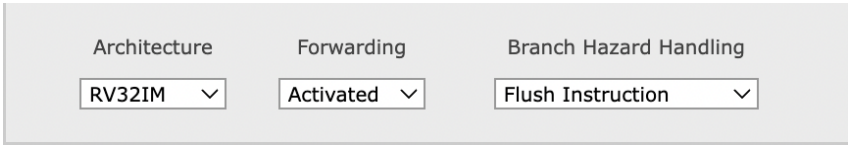
Data hazards between the following:

- addi t0, x0, 1 → sw t0, 8(sp): t0 is set in addi but needs to be used in sw.
- addi t0, x0, 2 → sw t0, 4(sp): t0 is set in addi but needs to be used in sw.
- addi t0, x0, 3 → sw t0, 0(sp): t0 is set in addi but needs to be used in sw.
- addi a1, x0, 3 → bge a2, a1, end: a1 is set in addi but needs to be checked in bge.
- lw t0, 0(sp) → add a0, a0, t0: t0 is loaded in lw but needs to be added in add.
- add a0, a0, t0 → bge a2, a1, end: a0 is updated in add but needs to be checked in bge.

3.2.3 [8 pts] Please execute the program using the WebRISC-V simulator by applying the following

Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

settings:



How many cycles would it take for the program to execute? Please: (1) include a screenshot of the execution table in your report; (2) explain how the hazards are resolved.

EXECUTION TABLE

FULL LOOPS

Instruction	CPU Cycles																																				
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38
addi sp, sp, -12	F	D	X	M	W																																	
addi t0, x0, 1		F	D	X	M	W																																
sw t0, 8(sp)			F	D	X	M	W																															
addi t0, x0, 2				F	D	X	M	W																														
sw t0, 4(sp)					F	D	X	M	W																													
addi t0, x0, 3						F	D	X	M	W																												
sw t0, 0(sp)							F	D	X	M	W																											
addi a1, x0, 3								F	D	X	M	W																										
addi a2, x0, 0									F	D	X	M	W																								
addi a0, x0, 0										F	D	X	M	W																							
jal x0, 8											F	D	X	M	W																						
bge a2, a1, 48												F																									
bge a2, a1, 48													F	D	X	M	W																				
lw t0, 0(sp)														F	D	X	M	W																		
addi sp, sp, 4															F	D	X	M	W																	
add a0, a0, t0																F	D	X	M	W																
addi a2, a2, 1																	F	D	X	M	W															
jal x0, -40																		F	D	X	M	W														
bge a2, a1, 48																				F	D	X	M	W												
lw t0, 0(sp)																					F	D	X	M	W											
addi sp, sp, 4																						F	D	X	M	W										
add a0, a0, t0																							F	D	X	M	W									
addi a2, a2, 1																								F	D	X	M	W								
jal x0, -40																									F	D	X	M	W							
bge a2, a1, 48																											F	D	X	M	W					
lw t0, 0(sp)																												F	D	X	M	W				
addi sp, sp, 4																													F	D	X	M	W			
add a0, a0, t0																														F	D	X	M	W		
addi a2, a2, 1																															F	D	X	M	W	
jal x0, -40																																F	D	X	M	W	
bge a2, a1, 48																																	F	D	X	M	W
lw t0, 0(sp)																																		F			

38 cycles and the Hazards are resolved with forwarding.